

HOJA DE RUTA DE REMEDIACIÓN

City2Cruise — Shop&Drop; Port Hub

Plan ejecutable derivado de la Auditoría Técnica Integral 2026-04-29

Documento	Hoja de ruta de remediación post-auditoría
Documento origen	AUDITORIA_TECNICA_INTEGRAL_2026-04-29.pdf
Cliente	REKER Tech Solutions S.L.
Proyecto	APP_TRASNORTE_LOCKERS_BARCELONA — fase 2-4
Rama destino	FASE4-FASE5-FASE6 (PR final hacia main)
Fecha	29 de abril de 2026
Capítulos	8 capítulos · 38 hitos · 14 hallazgos cubiertos
Esfuerzo total	~ 12 jornadas-persona (3 sprints de 1 semana)
Bloque inmediato	Capítulo 1 — antes de cualquier exposición pública
Verificación	Re-auditoría a 30 días (Capítulo 8)

Índice

Capítulo 0	Preparación y reglas del juego	Pág. 3
Capítulo 1	Bloque inmediato — pre-producción (5 hitos)	Pág. 4
Capítulo 2	Hardening corto plazo (4 hitos)	Pág. 8
Capítulo 3	Calidad de código y tipado (4 hitos)	Pág. 11
Capítulo 4	Modernización de dependencias (3 hitos)	Pág. 13
Capítulo 5	DevSecOps avanzado (4 hitos)	Pág. 14
Capítulo 6	Documentación y operación (3 hitos)	Pág. 16
Capítulo 7	Validación de carga y observabilidad (3 hitos)	Pág. 17
Capítulo 8	Re-auditoría y cierre formal (4 hitos)	Pág. 18
Anexo A	Resumen de hitos y dependencias	Pág. 19
Anexo B	Plantilla de checklist por hito	Pág. 20

Capítulo 0 — Preparación y reglas del juego

Antes de empezar, fijar la disciplina de trabajo para que cada hito sea verificable, trazable y reversible. Esto evita romper el estado actual del proyecto durante la remediación.

0.1 Estrategia de ramas (git flow ligero)

Crear una rama integradora `fix/audit-remediation-2026-04` desde FASE4-FASE5-FASE6. Cada hito vive en su propia feature-branch `fix/H-X.Y-slug` y se mergea a la integradora vía PR pequeño con conventional commit (*fix: / chore: / docs:*). Al cerrar todos los capítulos del bloque inmediato, se promueve la integradora a *main*.

0.2 Convenciones

- Cada PR debe pasar el CI completo (lint+tsc+jest+vitest+build+audit+docker-build).
- Cada PR enlaza al hito correspondiente en este documento (Closes #H-1.1).
- Cada hito que toque seguridad o configuración debe ir acompañado de una nota en `docs/devops/HITO_REMEDIACION_*.md` con evidencias (capturas de logs, salidas de comandos).
- Conventional commits (semantic-release ya está activo).

0.3 Pre-flight check (snapshot del estado actual)

Antes de tocar nada, generar evidencia del estado actual para poder comparar al final:

```
# Snapshot pre-remediación
cd backend && npm audit --json > /tmp/audit-backend-pre.json
cd ../cruise-connect-main && npm audit --json > /tmp/audit-frontend-pre.json
docker build -t city2cruise-backend:pre ./backend
docker build -t city2cruise-frontend:pre ./cruise-connect-main
docker scout cves city2cruise-backend:pre > /tmp/scout-backend-pre.txt
docker scout cves city2cruise-frontend:pre > /tmp/scout-frontend-pre.txt
git tag pre-remediation-2026-04-29
git push origin pre-remediation-2026-04-29
```

0.4 Criterio global de finalización del programa

El programa se considera **completado** cuando:

- Los 38 hitos están marcados como cerrados con su PR mergeado.
- `npm audit --audit-level=high` retorna 0 hallazgos en backend y frontend.
- ZAP baseline (workflow zap-baseline.yml) en verde sobre staging real.
- Re-auditoría a 30 días (Capítulo 8) firmada.

Capítulo 1 — Bloque inmediato (pre-producción)

Cinco hitos que cubren los hallazgos altos S-01 a S-04 y S-07. **Ninguno de ellos puede quedar abierto cuando la app reciba tráfico real.** Esfuerzo conjunto: ~ 1 jornada.

H-1.1 — Eliminar fallbacks hardcodeados de VAPID

ALTO · S-01

Objetivo	Aplicar el patrón fail-fast en producción a las claves VAPID y rotar la clave actualmente expuesta.
Ficheros	backend/src/config/env.ts · envs/production.env (real, fuera de git) · scripts/generate-vapid.sh (nuevo)
Esfuerzo	1 hora
Owner	Backend lead
Depende de	Capítulo 0 (snapshot)

Pasos detallados:

1) Generar par de claves nuevo:

```
# scripts/generate-vapid.sh
npx web-push generate-vapid-keys --json > /tmp/vapid.json
echo "Public: $(jq -r .publicKey /tmp/vapid.json)"
echo "Private: $(jq -r .privateKey /tmp/vapid.json)"
```

2) Refactorizar backend/src/config/env.ts sustituyendo el fallback hardcoded por el patrón fail-fast equivalente al de JWT_SECRET:

```
vapid: {
  publicKey: (() => {
    const k = process.env.VAPID_PUBLIC_KEY;
    if (!k && process.env.NODE_ENV === 'production')
      throw new Error('FATAL: VAPID_PUBLIC_KEY obligatorio en produccion');
    return k || ''; // dev acepta vacío
  })(),
  privateKey: (() => {
    const k = process.env.VAPID_PRIVATE_KEY;
    if (!k && process.env.NODE_ENV === 'production')
      throw new Error('FATAL: VAPID_PRIVATE_KEY obligatorio en produccion');
    return k || '';
  })(),
  subject: process.env.VAPID_SUBJECT || 'mailto:admin@city2cruise.com',
},
```

3) Actualizar envs/production.env y envs/staging.env reales con las claves nuevas. Sustituir el secreto en Fly: fly secrets set VAPID_PUBLIC_KEY=... VAPID_PRIVATE_KEY=... --app city2cruise-backend.

4) Rotación: invalidar las claves antiguas. Como las suscripciones de los usuarios están atadas a la clave pública, comunicar en release notes que los clientes recibirán un *pushManager.subscribe* nuevo en su próxima sesión (el SW del frontend lo gestiona automáticamente al cambiar VAPID).

5) Añadir test que verifique el throw:

```
// backend/src/__tests__/env.test.ts
test('falla en produccion sin VAPID_PRIVATE_KEY', () => {
  process.env.NODE_ENV = 'production';
  delete process.env.VAPID_PRIVATE_KEY;
  expect(() => require('../config/env')).toThrow(/VAPID_PRIVATE_KEY/);
});
```

Criterio de aceptación:

- `grep -r 'BGscsMy01ynE\|0XNrTZGcD0' backend/src/` retorna vacío.
- Test `env.test.ts` en verde.
- Despliegue staging arranca con las nuevas VAPID keys; un test e2e de push (envío real) sigue funcionando.

H-1.2 — Cerrar vulnerabilidades altas en frontend (axios, lodash)

ALTO · S-02

Objetivo	0 advisories HIGH en `npm audit` del frontend, sin romper API.
Ficheros	cruise-connect-main/package.json · cruise-connect-main/package-lock.json
Esfuerzo	1-3 horas (incluye verificación de regresiones)
Owner	Frontend lead
Depende de	—

Pasos detallados:

1) Snapshot inicial (ya hecho en Cap. 0). Anotar versión actual:

```
cd cruise-connect-main
npm ls axios lodash | tee /tmp/deps-pre.txt
```

2) Aplicar fix automático seguro:

```
npm audit fix
# Si quedan transitivas:
npm audit fix --force --dry-run # revisar breaking changes
# Cuando haya certeza, sin --dry-run.
```

3) Si aún quedan transitivas (típicamente lodash de algún Radix antiguo), añadir overrides a package.json (ya hay un patrón en el repo):

```
"overrides": {
  "serialize-javascript": "7.0.5",
  "vite": "5.4.21",
  "vite-plugin-pwa": "0.20.5",
  "axios": "^1.15.0",
  "lodash": "^4.17.24"
}
```

4) Validar build y tests:

```
npm ci --legacy-peer-deps
npm run lint
npx tsc --noEmit
npm test
npm run build
npm run e2e # si playwright esta configurado en local
```

5) Comparar bundle size con `npm run build:analyze` (rollup-visualizer ya añadido) y verificar que el cambio no infla el bundle más de un 5%.

Criterio de aceptación:

- `npm audit --audit-level=high` retorna 0 *vulnerabilities*.
- Tests vitest + playwright en verde.
- Bundle entry sigue por debajo del baseline 31.6 KB (gz 11 KB) +5%.

H-1.3 — Contenedores no-root + HEALTHCHECK por imagen

ALTO · S-03

Objetivo	Reducir blast radius si hay RCE: que ningún proceso contenedorizado corra como UID 0.
Ficheros	backend/Dockerfile · cruise-connect-main/Dockerfile · cruise-connect-main/nginx.conf (puerto 8080)
Esfuerzo	2 horas
Owner	DevOps / Backend
Depende de	—

Pasos backend (la imagen *node:20-alpine* ya incluye el usuario *node*):

```
# Stage 2: Runtime
FROM node:20-alpine AS runtime
WORKDIR /app
COPY --chown=node:node package*.json ./
RUN npm ci --omit=dev
COPY --from=builder --chown=node:node /app/dist ./dist
USER node
EXPOSE 9000
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
CMD wget --spider -q http://localhost:9000/api/health || exit 1
CMD ["node", "dist/index.js"]
```

Pasos frontend (Nginx unprivileged: imagen oficial *nginxinc/nginx-unprivileged* escucha en 8080):

```
# Stage 2: Runtime
FROM nginxinc/nginx-unprivileged:alpine AS runtime
USER 0
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
USER 101
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
CMD wget --spider -q http://localhost:8080/ || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

Importante: cambiar `listen 80` por `listen 8080` en *nginx.conf* y actualizar *docker-compose.yml* y los healthchecks para apuntar al nuevo puerto interno (publicar 80→8080).

Criterio de aceptación:

- `docker run --rm img id -u` retorna distinto de 0 en ambas imágenes.
- `docker inspect city2cruise-backend:ci --format '{{.Config.Healthcheck}}'` contiene HEALTHCHECK.
- `docker-compose up` sigue funcionando end-to-end (health verde en 60 s).

H-1.4 — Cabeceras de seguridad en Nginx (frontend)

MEDIO · S-04

Objetivo	Que la SPA servida por Nginx exponga HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer
Ficheros	cruise-connect-main/nginx.conf
Esfuerzo	1 hora (CSP requiere ajuste fino)
Owner	Frontend / DevOps
Depende de	H-1.3 (cambio de puerto 8080)

Configuración propuesta:

```
# Headers de seguridad (Hito H-1.4)
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
add_header Permissions-Policy "camera=(), microphone=(), geolocation=(self),
payment=(self)" always;
add_header Content-Security-Policy
"default-src 'self';
script-src 'self' 'wasm-unsafe-eval' https://js.stripe.com;
style-src 'self' 'unsafe-inline';
img-src 'self' data: blob: https://*.tile.openstreetmap.org https://tile.openstreetmap.org;
font-src 'self' data:;
connect-src 'self' https://api.stripe.com wss: ws: ${API_URL} ${SOCKET_URL}
https://*.sentry.io;
frame-src https://js.stripe.com https://hooks.stripe.com;
worker-src 'self' blob:;
manifest-src 'self';
object-src 'none';
base-uri 'self';
form-action 'self';" always;
```

Notas: *'unsafe-inline'* en *style-src* es necesario para Tailwind/shadcn; *'wasm-unsafe-eval'* lo requiere recharts. Para *script-src*, comprobar con SecurityHeaders.com que no rompe el SDK de Sentry; si lo hace, añadir <https://browser.sentry-cdn.com>.

Validación:

```
curl -I https://staging.city2cruise.es/ | grep -E '^(strict-transport|content-security|x-)'
```

Criterio de aceptación:

- Test manual con <https://securityheaders.com/?q=staging.city2cruise.es> nota A ($\geq$ A-).
- ZAP baseline workflow no añade nuevos warnings de cabeceras.
- La SPA carga sin errores CSP en DevTools.

H-1.5 — Limpiar IP local hardcodeada en logs de arranque

MEDIO · S-07

Objetivo	No filtrar la IP del equipo de desarrollador en logs de producción.
Ficheros	backend/src/index.ts
Esfuerzo	10 minutos
Owner	Backend
Depende de	—

```
// backend/src/index.ts (línea 36 actual)
- logger.info({ url: `http://192.168.1.47:${config.port}/api/health` }, 'Health check');
+ logger.info({ port: config.port, path: '/api/health' }, 'Health check disponible');
```

Criterio: grep -rn '192\168\1\47' backend/src sin coincidencias.

Capítulo 2 — Hardening corto plazo

Cuatro hitos que endurecen el sistema sin urgencia operativa pero antes de la primera ola de tráfico orgánico. Esfuerzo conjunto: ~ 2 jornadas.

H-2.1 — Reforzar política CORS y alertar rechazos

MEDIO · S-06

Objetivo	Sustituir <code>startsWith</code> permisivo por whitelisting explícito + observabilidad de rechazos.
Ficheros	backend/src/server.ts · backend/src/sockets/io.ts
Esfuerzo	2 horas
Owner	Backend
Depende de	—

Pasos:

- 1) Construir un set explícito de orígenes permitidos por entorno desde la configuración:

```
// backend/src/config/env.ts (añadir)
allowedOrigins: (process.env.ALLOWED_ORIGINS ||
'http://localhost:9100,http://localhost:9101,http://localhost:9102')
.split(',').map(s => s.trim()).filter(Boolean),
```

- 2) Reemplazar la lambda de cors por una validación contra el set:

```
const allow = new Set(config.allowedOrigins);
const localPortRegex = /^http:\/\/localhost:(\d{4,5})$/;
const lanRegex = /^http:\/\/192\.168\.d+\.d+(:\d+)?$/;

app.use(cors({
  credentials: true,
  origin(origin, cb) {
    if (!origin) return cb(null, true);
    if (allow.has(origin)) return cb(null, true);
    if (process.env.NODE_ENV !== 'production' && localPortRegex.test(origin)) return cb(null, true);
    if (process.env.NODE_ENV !== 'production' && lanRegex.test(origin)) return cb(null, true);
    Sentry.captureMessage('CORS rechazo', { level: 'warning', extra: { origin } });
    cb(new Error('Not allowed by CORS'));
  },
}));
```

- 3) Replicar idéntica lógica en `sockets/io.ts`.

- 4) Añadir test de regresión:

```
// backend/src/__tests__/cors.test.ts
test('rechaza origin foraneo', async () => {
  const res = await request(app).get('/api/health').set('Origin',
'https://attacker.example');
  expect(res.headers['access-control-allow-origin']).toBeUndefined();
});
```

Criterio: test `cors.test.ts` verde, alerta en Sentry cuando algún origen no permitido pega.

H-2.2 — Migrar `console.log` al logger estructurado

BAJO · S-08

Objetivo	0 <code>console.log</code> en src/ de producción; activar regla ESLint preventiva.
Ficheros	backend/src/**/*.ts · cruise-connect-main/src/**/*.ts,tsx · eslint.config.js

Esfuerzo	2 horas
Owner	Cualquiera (codemod)
Depende de	—

Pasos:

1) Inventario:

```
grep -rn 'console.log' backend/src cruise-connect-main/src \
--include='*.ts' --include='*.tsx' \
| grep -v '___tests__' | grep -v '/test/' > /tmp/console-log.txt
wc -l /tmp/console-log.txt # debe acabar en 0
```

2) Reemplazo automático con guía manual: en backend usar `logger.info/.debug`; en frontend envolver en helper `logger` que solo log si `import.meta.env.DEV` + reportar a Sentry en prod.

3) Añadir regla ESLint:

```
// eslint.config.js (frontend)
rules: {
  'no-console': ['warn', { allow: ['warn', 'error'] }],
  ...
}
```

4) Crear helper compartido frontend:

```
// cruise-connect-main/src/utils/logger.ts
export const logger = {
  debug: (...args: unknown[]) => { if (import.meta.env.DEV) console.debug(...args); },
  info: (...args: unknown[]) => { if (import.meta.env.DEV) console.info(...args); },
  warn: (...args: unknown[]) => console.warn(...args),
  error: (err: unknown, extra?: Record<string, unknown>) => {
    Sentry.captureException(err, { extra });
    if (import.meta.env.DEV) console.error(err, extra);
  },
};
```

Criterio: 0 console.log; lint en verde sin warnings de no-console.

H-2.3 — Limpieza de archivos residuales del repo

BAJO · S-09

Objetivo	Eliminar artefactos de migración y reorganizar scripts sueltos.
Ficheros	cruise-connect-main/package.json.backup · cruise-connect-main/{fix_vulnerabilities,start_all}.sh
Esfuerzo	10 minutos
Owner	Cualquiera
Depende de	H-1.2 (no perder overrides añadidos)

```
git rm cruise-connect-main/package.json.backup
git mv cruise-connect-main/start_all.sh scripts/dev-start-all.sh
git mv cruise-connect-main/fix_vulnerabilities.sh scripts/audit/fix-frontend-vulns.sh
chmod +x scripts/dev-start-all.sh scripts/audit/fix-frontend-vulns.sh
git commit -m "chore(repo): clean residual scripts and backup files"
```

Criterio: repo sin `*.backup` y los `.sh` dentro de `scripts/`.

H-2.4 — ESLint dedicado al backend con no-floating-promises

INFO · I-01

Objetivo	Detectar promesas sin <code>await/catch</code> y otros bugs de tipos en el backend.
-----------------	---

Ficheros	backend/.eslintrc.cjs · backend/package.json (script lint)
Esfuerzo	2 horas
Owner	Backend
Depende de	—

```
// backend/.eslintrc.cjs
module.exports = {
  root: true,
  parser: '@typescript-eslint/parser',
  parserOptions: {
    project: './tsconfig.json',
    tsconfigRootDir: __dirname,
  },
  plugins: ['@typescript-eslint'],
  extends: [
    'eslint:recommended',
    'plugin:@typescript-eslint/recommended',
    'plugin:@typescript-eslint/recommended-type-checked',
  ],
  rules: {
    '@typescript-eslint/no-floating-promises': 'error',
    '@typescript-eslint/no-misused-promises': 'error',
    '@typescript-eslint/no-explicit-any': 'warn',
    'no-console': ['error', { allow: ['warn', 'error'] }],
  },
  ignorePatterns: ['dist', 'node_modules', 'src/__tests__'],
};
```

Añadir al CI ([.github/workflows/ci.yml](#)) en el job backend, antes de tsc:

```
- name: Lint (eslint)
  run: npx eslint src --max-warnings=0
```

Criterio: CI backend con paso de lint en verde y warnings tope 0.

Capítulo 3 — Calidad de código y tipado

Cuatro hitos para reducir la deuda técnica del frontend (S-10) sin parar el desarrollo. Plan progresivo en 3 sprints. Esfuerzo total ~ 3 jornadas distribuidas.

H-3.1 — Activar strict en tsconfig.app.json (modo "transition")

MEDIO · S-10 (1/4)

Objetivo	Activar strict pero permitir transición — error si compila pero warn-only en módulos legacy.
Ficheros	cruise-connect-main/tsconfig.app.json
Esfuerzo	1 jornada
Owner	Frontend
Depende de	—

Estrategia: activar primero los flags más fáciles (strictBindCallApply, strictFunctionTypes), luego strictNullChecks, y al final noImplicitAny. Cada flag genera un set de errores que se arregla en su propio commit.

```
// tsconfig.app.json (paso 1)
"strict": true,
"noImplicitAny": false, // se activara en H-3.3
"strictNullChecks": false, // se activara en H-3.2
"strictFunctionTypes": true,
"strictBindCallApply": true,
"alwaysStrict": true,
"useUnknownInCatchVariables": true
```

Criterio: npx tsc --noEmit sin errores.

H-3.2 — Activar strictNullChecks

MEDIO · S-10 (2/4)

Objetivo	Tipar correctamente los nullables — el mayor cazador de bugs en SPAs React.
Esfuerzo	1 jornada · iterativo
Owner	Frontend
Depende de	H-3.1

Activar strictNullChecks: true y arreglar los errores en orden:

- Componentes UI (shadcn/ui ya está tipado correctamente).
- Hooks personalizados.
- Pages (DriverDashboard, ClientDashboard, AdminDashboard, ControlTowerPage).
- Services / api wrappers (suelen ser los que más nullables tocan).

Patrón típico de fix:

```
// Antes
const user = data.user;
if (user.id) ...

// Despues
const user = data?.user;
if (user?.id) ...
```

Criterio: tsc en verde, sin !. (non-null assertion) excepto justificadas.

H-3.3 — Erradicar `any` y activar noImplicitAny

MEDIO · S-10 (3/4)

Objetivo	Eliminar las 45 ocurrencias de any y prevenir nuevas.
Esfuerzo	1 jornada
Owner	Frontend
Depende de	H-3.2

```
# Inventario
grep -rn ': any\|<any>\|as any' cruise-connect-main/src \
--include='*.ts' --include='*.tsx' | grep -v __tests__ | tee /tmp/any.txt
```

Sustituir por:

- unknown + type guards cuando viene de la API y se valida con Zod.
- Tipos generados desde el backend (compartir tipos vía un paquete `@city2cruise/api-types` o tipar manualmente en `src/types/`).
- Genéricos donde aplique.

Activar regla ESLint:

```
'@typescript-eslint/no-explicit-any': 'error',
```

Criterio: 0 ocurrencias de any en src/ (excluyendo tests con justificación).

H-3.4 — Compartir tipos backend ↔ frontend

INFO

Objetivo	Eliminar el drift entre el contrato del backend y los tipos del cliente.
Ficheros	Nueva carpeta <code>packages/api-types/</code> o uso de Zod schemas exportados
Esfuerzo	0.5 jornada
Owner	Full-stack
Depende de	H-3.3

Opción A (recomendada): exportar los `z.infer<typeof Schema>` del backend (ya hay Zod) como un sub-paquete y consumirlo en frontend.

Opción B: generar tipos OpenAPI con `tsoa` u `openapi-typescript` a partir de un spec generado por las rutas.

Criterio: contratos GET `/api/lockers`, POST `/api/requests`, etc. tipados desde una sola fuente.

Capítulo 4 — Modernización de dependencias

Cierre del último advisory que queda tras Cap. 1 (S-05) y plan recurrente para que no se vuelva a acumular deuda. Esfuerzo: ~ 0.5 jornada + recurrente.

H-4.1 — Actualizar Vite ≥ 6 y vite-plugin-pwa compatible

MEDIO · S-05

Objetivo	Cerrar GHSA-67mh-4ww8-2f99 (esbuild dev-server CORS).
Esfuerzo	0.5 jornada (validar plugin PWA)
Owner	Frontend
Depende de	Release upstream de vite-plugin-pwa compatible con vite 6

Pasos:

- 1) Verificar release notes de vite-plugin-pwa: `npm view vite-plugin-pwa versions`.
- 2) Actualizar overrides en package.json.
- 3) `npm i vite@^6 vite-plugin-pwa@^X --save-dev --legacy-peer-deps`.
- 4) Validar que el manifest.webmanifest se sigue generando y que workbox precachea correctamente.
- 5) Re-run npm audit.

Criterio: 0 advisories MODERATE.

H-4.2 — Activar Dependabot / Renovate

INFO

Objetivo	Que las próximas vulnerabilidades aparezcan como PRs automáticos.
Ficheros	.github/dependabot.yml o .github/renovate.json
Esfuerzo	30 minutos

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/cruise-connect-main"
    schedule: { interval: "weekly" }
    open-pull-requests-limit: 5
    labels: ["deps", "frontend"]
  - package-ecosystem: "npm"
    directory: "/backend"
    schedule: { interval: "weekly" }
    open-pull-requests-limit: 5
    labels: ["deps", "backend"]
  - package-ecosystem: "docker"
    directory: "/backend"
    schedule: { interval: "monthly" }
  - package-ecosystem: "docker"
    directory: "/cruise-connect-main"
    schedule: { interval: "monthly" }
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule: { interval: "monthly" }
  - package-ecosystem: "terraform"
    directory: "/terraform/flyneonupstash"
    schedule: { interval: "monthly" }
  - package-ecosystem: "pip"
    directory: "/digital_twin"
```

```
schedule: { interval: "weekly" }
```

Criterio: primer PR automático llega en menos de 7 días.

H-4.3 — Política escrita de manejo de CVEs

INFO

Objetivo	Que el equipo entienda cuándo bloquear el merge y cuándo se acepta como deuda.
Ficheros	docs/devops/SECURITY_POLICY.md
Esfuerzo	30 minutos

Template breve: critical → bloquea CI y release · high → 7 días · moderate → 30 días · low → próximo sprint.
Owner del proceso: backend lead. Excepciones documentadas en GitHub Issue con etiqueta *security-debt* y caducidad obligatoria.

Capítulo 5 — DevSecOps avanzado

Tres mejoras opcionales pero recomendadas que llevan al proyecto a un nivel *enterprise-grade*: SBOM, firma de imágenes, escaneo de runtime. Esfuerzo: ~ 1 jornada.

H-5.1 — SBOM CycloneDX y Trivy en CI

INFO · I-02

Objetivo	Inventario verificable de dependencias y escaneo de vulns sobre la imagen final.
Ficheros	.github/workflows/cd.yml
Esfuerzo	3 horas

```
- name: Generate SBOM (backend)
uses: anchore/sbom-action@v0
with:
image: ghcr.io/${{ github.repository_owner }}/city2cruise-backend:${{
steps.meta_b.outputs.version }}
format: cyclonedx-json
output-file: sbom-backend.cdx.json
upload-artifact: true

- name: Trivy scan (backend image)
uses: aquasecurity/trivy-action@master
with:
image-ref: ghcr.io/${{ github.repository_owner }}/city2cruise-backend:${{
steps.meta_b.outputs.version }}
format: sarif
output: trivy-backend.sarif
severity: CRITICAL,HIGH
exit-code: 1
```

Criterio: SBOM publicado como artifact en cada release; Trivy bloquea el deploy si hay HIGH/CRITICAL.

H-5.2 — Firma de imágenes con cosign (sigstore)

INFO · I-02

Objetivo	Garantizar que solo se despliegan imágenes firmadas por la pipeline oficial.
Esfuerzo	3 horas

```
- uses: sigstore/cosign-installer@v3
- name: Sign image (keyless OIDC)
env:
COSIGN_EXPERIMENTAL: "true"
run: |
cosign sign --yes ghcr.io/${{ github.repository_owner }}/city2cruise-backend@${{
steps.build.outputs.digest }}
cosign sign --yes ghcr.io/${{ github.repository_owner }}/city2cruise-frontend@${{
steps.build.outputs.digest }}
- name: Verify (smoke)
run: cosign verify --certificate-identity-regexp '.*' --certificate-oidc-issuer-regexp '.*'
\
ghcr.io/.../city2cruise-backend@${{ steps.build.outputs.digest }}
```

Configurar Fly.io para que rechace imágenes sin firma válida (mediante un init-container de validación o policy en CD).

Criterio: cosign verify verde sobre la imagen publicada.

H-5.3 — Branch protection rules formales en main y release

INFO

Objetivo	Hacer cumplir el requisito de PRs con CI verde + reviews.
Ficheros	Configuración de GitHub (no es código)
Esfuerzo	30 minutos

Reglas mínimas en *main*: 1 review aprobada, status checks obligatorios (frontend, backend, security, docker-build, digital-twin, rl-service-bridge, e2e), linear history, no force push, no eliminar rama. Documentar en docs/devops/HITO_5_1_3 y verificar contra los settings reales.

Criterio: intento de force-push o merge sin review es rechazado.

H-5.4 — Secret scanning en pre-commit (gitleaks local)

INFO

Objetivo	Prevenir commits con secretos antes de que lleguen al CI.
Esfuerzo	1 hora

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.0
  hooks:
  - id: gitleaks
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.5.0
  hooks:
  - id: detect-private-key
  - id: end-of-file-fixer
  - id: trailing-whitespace
```

Documentar instalación: `pip install pre-commit && pre-commit install`.

Capítulo 6 — Documentación y operación

Tres hitos para limpiar y consolidar la documentación. No bloquean el código pero hacen que el proyecto sea auditable de un vistazo. Esfuerzo: ~ 0.5 jornada.

H-6.1 — Reorganizar documentación histórica

INFO · I-03

Objetivo	Que la raíz contenga solo los documentos vivos y mover el resto a docs/history/.
Ficheros	docs/history/ (nueva)
Esfuerzo	30 minutos

```
mkdir -p docs/history
git mv AUDITORIA_TECNICA_CITY2CRUISE.md
docs/history/AUDITORIA_TECNICA_CITY2CRUISE_v1_2026-03-17.md
git mv RE_AUDITORIA_TECNICA_CITY2CRUISE_v2.md docs/history/
git mv RESPUESTA_CONSULTAS_TECNICAS.md docs/history/
git mv PLAN_EJECUCION_v2.md docs/history/
git mv PLAN_EJECUCION_V3_MIGRACIONSQLITE.md docs/history/
git mv PLAN_EJECUCION_AUDITORIA.md docs/history/
git commit -m "docs: archive historical audits and plans into docs/history"
```

Mantener en raíz: README.md (nuevo, derivado del actual), HOJA_DE_RUTA_REMEDIACION_2026-04-29.pdf, AUDITORIA_TECNICA_INTEGRAL_2026-04-29.pdf, MEMORIA_TECNICA.md (en cruise-connect-main), Makefile, .gitignore.

H-6.2 — README de raíz reescrito (índice del monorepo)

INFO

Objetivo	Que un nuevo desarrollador entienda el monorepo en 3 minutos.
Ficheros	README.md (raíz)
Esfuerzo	1 hora

Estructura recomendada: 1) Visión, 2) Arquitectura (diagrama Mermaid), 3) Estructura del repo, 4) Quickstart (clone, env, docker-compose up), 5) Enlaces a docs/ (HITOs, ADR, runbooks, audits), 6) Contributing (commit convention), 7) Licencia.

H-6.3 — Diagrama de arquitectura actualizado

INFO

Objetivo	Diagrama oficial actual coincide con el código real.
Ficheros	docs/architecture.mmd · docs/architecture.png
Esfuerzo	1 hora

Componentes a representar: SPA React (CF Pages) → Nginx → Express + Socket.IO → PostgreSQL (Neon) + Redis (Upstash) ■ rl_service ■ digital_twin. Lateral: Sentry, Prometheus, Grafana, Stripe, Twilio, web-push.

Capítulo 7 — Validación de carga y observabilidad

Tres hitos para confirmar que las decisiones de capacidad (memory_mb=512 en Fly, min_machines=2, hard_limit=500) aguantan tráfico real. Esfuerzo: ~ 1 jornada.

H-7.1 — Ejecutar suite k6 contra staging

INFO · I-04

Objetivo	Confirmar SLOs (p95 latencia, error rate, saturación) bajo carga simulada.
Ficheros	k6/phase4-100c.js · k6/phase4-spike.js · scripts/k6-phase4.sh
Esfuerzo	0.5 jornada

```
# Ejecutar contra staging desplegado
K6_OUT=cloud BASE_URL=https://staging.city2cruise.es ./scripts/k6-phase4.sh
# Generar dashboard html exportable
k6 run --out json=k6-results.json k6/phase4-100c.js
```

Métricas a vigilar: http_req_duration p95 < 500 ms, http_req_failed < 1%, process_resident_memory_bytes en Fly < 380 MB (margen sobre 512).

Criterio: dashboard k6 publicado en docs/audits/, sin OOM ni 5xx en staging.

H-7.2 — Validar alertas Prometheus + Sentry

INFO

Objetivo	Comprobar que las alertas saltan cuando deben.
Ficheros	observability/alert-rules.yml
Esfuerzo	2 horas

Inducir condiciones para cada alert-rule: forzar 5xx (provocar excepción no manejada), saturar rate-limit, parar Redis temporalmente, etc. Confirmar que llega notificación al canal correspondiente y que la severidad y descripción son correctas.

H-7.3 — Plan DR ensayado (game day)

INFO

Objetivo	Validar el runbook DR con un ensayo controlado en staging.
Ficheros	docs/runbooks/DR_RUNBOOK.md
Esfuerzo	0.5 jornada (incluye write-up post-mortem)

Escenario: simular pérdida de la base Neon → restaurar desde PITR de la noche anterior. Cronómetro RTO/RPO. Documentar mejoras en el runbook y crear PR.

Capítulo 8 — Re-auditoría y cierre formal

Cuatro hitos finales para certificar que la remediación es real y queda evidencia.

H-8.1 — Re-ejecutar npm audit + ZAP baseline

Objetivo	Comparar contra el snapshot pre-remediación del Cap. 0.
Esfuerzo	30 minutos

```
cd backend && npm audit --json > /tmp/audit-backend-post.json
cd ../cruise-connect-main && npm audit --json > /tmp/audit-frontend-post.json
diff <(jq .metadata.vulnerabilities /tmp/audit-backend-pre.json) \
<(jq .metadata.vulnerabilities /tmp/audit-backend-post.json)
```

Disparar el workflow `zap-baseline.yml` contra staging real (no localhost).

H-8.2 — Penetration test ligero (pentest interno)

Objetivo	Validar lo que las herramientas automáticas no ven (lógica de negocio).
Esfuerzo	1 jornada

Casos mínimos: bypass de RBAC (DRIVER intentando endpoints CLIENT/ADMIN), session-fixation con refresh-token, replay de webhooks Stripe, IDOR en `/api/requests/:id`, re-uso de OTP de handshake, spoofing GPS más allá de lo testeado en `GpsValidationService`.

H-8.3 — Documento de cierre — Hito 6.6 Cumplimiento

Objetivo	Capturar evidencia formal de que cada hallazgo está cerrado.
Ficheros	<code>docs/devops/HITO_6_6_REMEDIACION_AUDITORIA.md</code>
Esfuerzo	1 hora

Una sección por hallazgo (S-01..I-04) con: problema · acción tomada · PR/commit · evidencia. Adjuntar diff de `npm audit` antes/después y capturas de `SecurityHeaders.com`.

H-8.4 — Tag, release y comunicación

Objetivo	Cerrar el ciclo: release semántico vinculado al cierre de la remediación.
Esfuerzo	30 minutos

```
git checkout main
git pull
git tag -a v1.0.0 -m "Release post-remediation audit 2026-04-29"
git push origin v1.0.0
# semantic-release se encargara del CHANGELOG y release notes
```

Comunicar al stakeholder (REKER Tech Solutions) con el PDF AUDITORIA + esta hoja de ruta + el documento de cierre H-8.3 + URL de la release.

Anexo A — Resumen de hitos y dependencias

Hito	Hallazgo	Sev.	Esfuerzo	Depende de	Criterio breve
H-1.1	S-01	ALTO	1 h	Cap. 0	VAPID guard + rotación
H-1.2	S-02	ALTO	1-3 h	—	0 advisories HIGH frontend
H-1.3	S-03	ALTO	2 h	—	USER no-root + HEALTHCHECK
H-1.4	S-04	MEDIO	1 h	H-1.3	Headers de seguridad nginx
H-1.5	S-07	MEDIO	10 min	—	Sin IP hardcodeada
H-2.1	S-06	MEDIO	2 h	—	CORS allow-list + alerta
H-2.2	S-08	BAJO	2 h	—	0 console.log; ESLint regla
H-2.3	S-09	BAJO	10 min	H-1.2	Repo limpio
H-2.4	I-01	INFO	2 h	—	ESLint backend en CI
H-3.1	S-10	MEDIO	1 d	—	tsconfig strict (transición)
H-3.2	S-10	MEDIO	1 d	H-3.1	strictNullChecks en verde
H-3.3	S-10	MEDIO	1 d	H-3.2	0 any en src/ frontend
H-3.4	—	INFO	0.5 d	H-3.3	Tipos compartidos backend↔front
H-4.1	S-05	MEDIO	0.5 d	Upstream	Vite 6 + plugin-pwa
H-4.2	—	INFO	30 min	—	Dependabot/Renovate activo
H-4.3	—	INFO	30 min	—	SECURITY_POLICY.md
H-5.1	I-02	INFO	3 h	H-1.3	SBOM + Trivy en CI
H-5.2	I-02	INFO	3 h	H-5.1	Imágenes firmadas con cosign
H-5.3	—	INFO	30 min	—	Branch protection main
H-5.4	—	INFO	1 h	—	pre-commit + gitleaks local
H-6.1	I-03	INFO	30 min	—	docs/history reorganizado
H-6.2	—	INFO	1 h	H-6.1	README de raíz nuevo
H-6.3	—	INFO	1 h	—	Diagrama arquitectura actual
H-7.1	I-04	INFO	0.5 d	Cap. 1	k6 verde sobre staging
H-7.2	—	INFO	2 h	Cap. 1	Alertas validadas
H-7.3	—	INFO	0.5 d	Cap. 1	Game day DR ensayado
H-8.1	Todos	INFO	30 min	Caps. 1-7	audit + ZAP comparados
H-8.2	Todos	INFO	1 d	H-8.1	Pentest interno OK
H-8.3	Todos	INFO	1 h	H-8.2	HITO_6_6 cerrado
H-8.4	—	INFO	30 min	H-8.3	v1.0.0 publicada

Anexo B — Plantilla de checklist por hito

Recomendamos copiar esta plantilla en cada PR de remediación para asegurar consistencia.

```
## H-X.Y — <título del hito>

### Contexto
- Hallazgo de auditoría: S-XX / I-XX
- Severidad original: ALTO/MEDIO/BAJO/INFO
- Documento: HOJA_DE_RUTA_REMEDIACION_2026-04-29.pdf · Cap. X

### Cambios introducidos
- [ ] Código (lista de ficheros)
- [ ] Configuración (envs, fly.toml, ...)
- [ ] Documentación (HITO_REMEDIACION_*, README)

### Verificación
- [ ] CI completo en verde
- [ ] Test específico añadido o cubierto por suite existente
- [ ] Comando manual ejecutado (output adjunto en /evidencia)
- [ ] Comparación con snapshot pre-remediación (si aplica)

### Riesgo y rollback
- Riesgo: <bajo/medio/alto y por qué>
- Rollback: <git revert SHA / fly secrets unset / ...>

### Evidencia
<adjuntar pegado de logs / capturas relevantes>

Closes: docs/HOJA_DE_RUTA · H-X.Y
```

— Fin de la hoja de ruta —